

On Wavelength

Calendrier de groupe avec heatmap voting et workflow complet sport-coach pour 8 sports.

Qu'est-ce que c'est

On Wavelength est une app calendrier iOS + Android avec deux piliers : heatmap voting pour rendez-vous de groupe (chacun tape ses créneaux libres, l'app choisit le meilleur, le chat reste silencieux) et un workspace sport-coach pour 8 sports couvrant entraînements, matchs, compositions, présences en direct et bibliothèque d'exercices.

À qui ça s'adresse

Deux audiences distinctes. Coaches amateurs sur les 8 sports + coaches de clubs de course (les meilleurs convertisseurs, 3-5 % vues->install). Colocs, groupes d'amis, familles, créateurs productivité (plus larges, ~0,5-1 % conversion). Un seul coach amène 8-16 utilisateurs actifs (son équipe), l'attribution coach se cumule ; la reach productivité garde le funnel plein entre campagnes coach.

Fonctionnalités clés

- Heatmap voting sur chaque groupe.
- Calendrier de groupe coloré avec événements partagés.
- Workspace équipe sportive : entraînements, matchs, compositions, présences, rappels.
- Bibliothèque d'exercices sur 8 sports, session-builder en 4 minutes.
- Planificateur de saison en 20 minutes.
- Multi-groupes : un coach avec plusieurs équipes + cercle privé coloc dans la même app.

Modèle économique

- Une seule source de revenu : abonnement Premium récurrent.
- Tier gratuit généreux : heatmap voting complet, un calendrier, un groupe jusqu'à 4 membres.
- Premium déverrouille : groupes illimités, workspace équipe, bibliothèque, présences, planificateur.
- Premium gratuit à vie pour affiliés onboarded, aucune obligation de publier.

Prix

- ~5 EUR / mois (pas de lock-in annuel).
- ~40 EUR / an (~ deux mois gratuits vs mensuel).
- ASC live depuis mai 2026 ; part affilié toujours 50 % du prix du moment.

Lacune du marché - ce qui manquait

Le heatmap voting pour rendez-vous de groupe est une lacune ouverte dans le marché des calendriers depuis des années. Les calendriers dominants (Google, Apple, Outlook) ne le proposent pas - trop niche pour un produit horizontal. Doodle existe mais vit comme URL-poll, en dehors d'un vrai calendrier. Côté sport-coach, TeamSnap et SportEasy dominent USA et partie d'Europe de l'Ouest, mais l'adoption en DACH et France est mince : TeamSnap est US-centric, SportEasy est tarifié pour clubs pas pour coaches amateurs. Deux audiences (groupes coordination-painé + coaches amateurs) qui achètent une calendar-app dès qu'elle résout leur douleur. On a construit un produit qui sert les deux.

Pourquoi maintenant

Deux changements structurels. D'abord, iOS Universal Links et Android install-referrer ont assez mûri pour que l'attribution DIY soit fiable (la pièce manquante - iOS cold - résolue avec un clipboard deferred-deeplink, vérifié live pendant des mois). Ça permet 50 % rev-share sans couche d'agence, ce qui n'était pas viable quand les SDK d'attribution coûtaient 0,20-0,40 USD par install. Ensuite, RevenueCat + pg_cron + Wise laissent un opérateur solo gérer une pipeline webhook-to-payout complète sans infrastructure payout managée.

Rémunération des affiliés

- 50% du revenu net des abonnements Premium pendant 24 mois à partir du premier mois payé de chaque utilisateur attribué.
- Retenue de 30 jours pour les remboursements avant que le paiement devienne réclamable (fenêtre de remboursement Apple / Google).
- Paiement minimum de 50 EUR, mensuel, via Wise.
- Premium gratuit à vie pour les affiliés onboarded (aucune obligation de publication).
- Tableau de bord par handle avec split en direct des clics / installs / abonnements / gains, sans couche d'agence.

Comment fonctionne le tracking - stack d'attribution DIY

La plupart des stacks affiliés tournent sur Branch.io, AppsFlyer ou un SDK d'attribution payant similaire. Nous avons construit le nôtre. Ces outils facturent par install attribué, ils ajoutent une tierce partie à l'expérience de premier lancement de l'utilisateur, et ils rendent les paiements aux influenceurs opaques. Nous voulions un contrôle total de chaque étape, du clic au virement Wise. Le chemin complet a six étapes - clic sur le lien, install, attribution au premier lancement, achat Premium, retenue, paiement. Voici ce qui tourne à chaque étape.

Pourquoi on a construit le nôtre au lieu de Branch.io

Trois raisons. (1) Coût : Branch.io et AppsFlyer facturent 0,20-0,40 USD par install attribué à notre niveau de volume, ce qui aurait absorbé environ 8-12 % de la cagnotte de paiements affiliés. Avec DIY on garde ça pour les affiliés. (2) Vie privée : chaque SDK supplémentaire lit le contexte de premier lancement - c'est un vendor de plus avec des données utilisateur first-touch qu'on ne voulait pas ajouter. Le framework App Tracking Transparency d'Apple a aussi rendu le fingerprinting probabiliste peu fiable depuis iOS 14.5 ; les SDK payants contournent ça avec des heuristiques opaques qu'on ne pouvait pas auditer. (3) Transparence : quand un chiffre du tableau de bord a l'air faux, on lit notre propre code en 10 minutes au lieu de déboguer une black-box tierce. Le trade-off c'est qu'on possède les bugs. Chaque ligne du code d'attribution est dans notre repo.

Android : Google Play install-referrer

Android nous donne le chemin le plus propre. Quand un utilisateur clique sur ton lien, l'install Google Play Store transporte la chaîne referrer originale (un paramètre query URL-encodé depuis la chaîne de redirection) jusqu'au premier lancement via Play Install Referrer API. Au premier lancement l'app lit ``getInstallReferrer().getInstallReferrer()``, on parse handle et source depuis là, et on écrit une ligne d'attribution dans notre table Supabase avec l'événement install. Déterministe. Pas de fingerprinting. Pas de presse-papiers.

iOS warm (app déjà installée)

Si l'utilisateur a déjà On Wavelength installé, iOS route le tap via un Universal Link directement dans l'app avec le handle dans le chemin URL. On enregistre ``onwavelength.space/i/*`` comme Universal Link dans le fichier d'entitlements iOS, le CDN d'Apple sert le JSON ``apple-app-site-association`` qu'on publie à la racine du domaine, l'OS route le tap. Dans l'app, le handler de deep-link lit le chemin URL, extrait le handle, écrit la ligne d'attribution. Déterministe.

iOS cold (~99 % du trafic créateur)

Le cas dur - et ~99 % du trafic créateur - c'est iOS cold install. iOS ne donne à l'app fraîchement installée aucun contexte sur comment elle a été atteinte. Tout le business de Branch.io est de résoudre ça avec des heuristiques de device-fingerprinting devenues de plus en plus peu fiables après iOS 14.5. Nous le faisons différemment. Quand un utilisateur clique sur ton lien dans Safari, notre chaîne de redirection touche `onwavelength.space/i/[handle]`. Cette page écrit un token JSON à courte durée de vie dans le presse-papiers avant de rediriger vers l'App Store. Le token contient : handle, timestamp (issued-at), app-key, version, et une signature HMAC-SHA256 calculée contre un secret côté serveur. Après install, au premier lancement, l'app lit le presse-papiers une fois (Apple montre l'UI système standard de paste-clipboard), parse le JSON, vérifie la signature, accepte le token s'il est signé frais et a moins de 7 jours, écrit la ligne d'attribution. L'utilisateur ne voit aucun écran supplémentaire quand ça marche.

Confirmation optionnelle au premier lancement

Fallback optionnel. Au premier ouverture, après qu'on a lu ce que le presse-papiers a fourni, l'app montre une seule ligne de confirmation : "Invité par @[handle-suggéré] ? Oui / Non". Le champ handle est pré-rempli avec ce que le presse-papiers a suggéré ; si le presse-papiers était vide, le champ est vide et l'utilisateur peut taper. La plupart des utilisateurs clique Oui. Ça attrape le cas limite où la dialogue d'accès au presse-papiers d'Apple a été refusé, l'utilisateur a copié autre chose entre le tap et l'install, ou le presse-papiers a expiré. L'étape est activable par app.

Webhook RevenueCat -> queue de paiement

Les achats Premium passent par RevenueCat. RC gère les reçus Apple StoreKit et Google Play Billing, le dedup, la détection de remboursement, et convertit tout en un format d'événement webhook cohérent. Quand un événement ``INITIAL_PURCHASE``, ``RENEWAL``, ``PRODUCT_CHANGE`` ou ``CANCELLATION`` tire, RC poste JSON à notre Edge function Supabase ``affiliate-rc-webhook``. La fonction (1) vérifie l'en-tête de signature RC contre notre secret partagé, (2) idempotency-check sur l'ID événement contre ``processed_webhook_events``, (3) cherche la ligne d'attribution de l'utilisateur, (4) calcule 50 % du revenu net après le cut Apple/Google de 15-30 %, (5) crédite la table ``affiliate_earnings`` avec handle, app, gross, cut, net, our-share, date holdback-until, et l'ID événement source. Le tout dans une transaction Postgres.

Pourquoi ça marche comme pitch affilié

Les coaches sportifs résolvent une douleur quotidienne (absences, chaos compositions, planification exercices) et convertissent au-dessus de la moyenne - et amènent leur équipe, un seul coach vaut 8-16 utilisateurs actifs. Audiences productivité/coloc convertissent moins mais la reach est bien plus large. Le heatmap voting se prête au screen-recording, donc même les créateurs faceless ont un asset visuel fort. La structure 50 % / 24 mois / Wise direct est inhabituelle pour une app de cette taille.

Pour les posts avec lien affilié : #ad / #sponsorisé / Paid Partnership (FTC, droit européen).